

Docker Desktop & Docker Hub — Hands-on Practical

1. Learning Objectives

By the end of this practical, students will be able to:

- Understand Docker and containerization
 - Install and configure Docker Desktop
 - Work with Docker CLI
 - Pull and run images from Docker Hub
 - Create and run containers
 - Build a Docker image using a **Dockerfile**
 - Containerize a React application
 - Tag Docker images
 - Push images to Docker Hub
 - Pull images from Docker Hub
 - Run a Docker Hub image locally
-

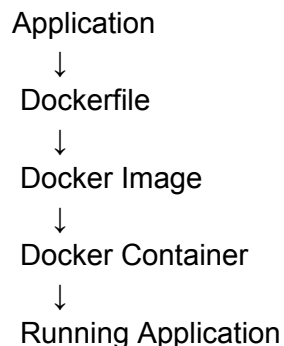
PART 1 — Docker Fundamentals

What is Docker?

Docker is a platform used to **build, package, and run applications inside lightweight containers**.

A container contains the application along with the dependencies required to run it.

Docker Workflow



Important Docker Terms

Term	Meaning
Docker	Platform for containerization
Image	Read-only package containing application and dependencies
Container	Running instance of an image
Dockerfile	Instructions used to build an image
Docker Hub	Cloud registry for storing and sharing images
Docker Desktop	Application providing Docker Engine and GUI on Windows/macOS
Registry	Repository for Docker images

PART 2 — Install Docker Desktop

Students should download Docker Desktop from the official Docker website:

[Docker Desktop](#)

Install Docker Desktop and restart the system if required.

After installation, open **PowerShell**.

Run:

```
docker --version
```

Expected output will look similar to:

```
Docker version XX.X.X
```

Then check Docker information:

```
docker info
```

If Docker Desktop is running correctly, Docker information should be displayed.

PART 3 — Run Your First Docker Container

We will use the Nginx image.

Step 1 — Download the image

```
docker pull nginx
```

This downloads the Nginx image from Docker Hub.

Step 2 — Check downloaded images

```
docker images
```

You should see:

```
nginx
```

Step 3 — Run the container

```
docker run -d -p 8080:80 nginx
```

Explanation:

```
-d    → Run container in background  
-p    → Map ports  
8080  → Host port  
80    → Container port  
nginx → Image
```

Step 4 — Check running containers

```
docker ps
```

You should see the Nginx container running.

Step 5 — Open the application

Open your browser:

```
http://localhost:8080
```

Students should see the **Nginx welcome page**.

PART 4 — Basic Docker Commands

View running containers

```
docker ps
```

View all containers

```
docker ps -a
```

Stop a container

First get the container ID:

```
docker ps
```

Then:

```
docker stop <container_id>
```

Example:

```
docker stop abc123
```

Remove a container

```
docker rm <container_id>
```

Remove an image

```
docker rm nginx
```

View Docker images

```
docker images
```

Docker Hub – Sharing and Running a Dockerized React Application

Objective

The objective of this practical is to understand how a Dockerized React application can be shared through Docker Hub and executed on another student's computer.

By completing this practical, students will learn:

- How to create a Dockerized React application
 - How to build a Docker image
 - How to create a Docker Hub repository
 - How to tag a Docker image
 - How to push an image to Docker Hub
 - How another student can pull the image
 - How to run the application on another computer
 - How Docker enables the concept of “**Build Once, Run Anywhere**”
-

Part 1 – Student A: Create the React Application

Student A will create a React application that will be shared with Student B.

Step 1 – Create a React Application

Open PowerShell.

Run:

```
npm create vite@latest docker-react-demo
```

Select:

Framework: React

Variant: JavaScript

Move into the project:

```
cd docker-react-demo
```

Install dependencies:

```
npm install
```

Part 2 – Test the React Application

Start the application:

```
npm run dev
```

The application will normally be available at:

```
http://localhost:5173
```

Open the URL in the browser and verify that the React application is working.

Stop the development server:

```
Ctrl + C
```

Part 3 – Create a Dockerfile

Inside the React project folder, create a file named:

Dockerfile

Add the following content:

```
FROM node:22-alpine AS builder
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
FROM nginx:alpine
```

```
COPY --from=builder /app/dist /usr/share/nginx/html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

This Dockerfile uses a **multi-stage build**.

Build Stage

Node.js



Install dependencies



Copy React project



Build React application



Generate dist folder

Production Stage

Nginx



Copy React build files



Serve React application

Part 4 – Build the Docker Image

Make sure you are inside the project directory:

```
cd docker-react-demo
```

Build the Docker image:

```
docker build -t reactapp .
```

Explanation:

`docker build` → Build a Docker image

`-t reactapp` → Give the image the name reactapp

`.` → Use the current directory as build context

Check the image:

```
docker images
```

You should see:

```
reactapp
```

Part 5 – Test the Docker Container

Run the Docker container:

```
docker run -d -p 8081:80 reactapp
```

Check the running container:

```
docker ps
```

Open the browser:

```
http://localhost:8081
```

The React application should now be running inside a Docker container.

Part 6 – Create a Docker Hub Account

Student A needs a Docker Hub account.

Open:

```
https://hub.docker.com/
```

Create an account or log in to an existing account.

Remember the Docker Hub username because it will be used when tagging the Docker image.

Example:

Docker Hub Username:
studentA

Part 7 – Login to Docker Hub

Open PowerShell.

Run:

```
docker login
```

Enter the required Docker Hub credentials.

If authentication is successful, you should see:

Login Succeeded

For better security, a Docker Hub Personal Access Token can be used instead of the account password.

Part 8 – Tag the Docker Image

The local image currently has the name:

reactapp

Docker Hub images use the following naming format:

<dockerhub-username>/<repository-name>:<tag>

Example:

studentA/reactapp:latest

Tag the image:

```
docker tag reactapp studentA/reactapp:latest
```

Replace **studentA** with the actual Docker Hub username.

Check the images:

```
docker images
```

You should see something similar to:

```
reactapp  
studentA/reactapp
```

Part 9 – Push the Image to Docker Hub

Push the image:

```
docker push studentA/reactapp:latest
```

Replace **studentA** with the actual Docker Hub username.

Docker will upload the image layers to Docker Hub.

After the push is completed, open:

<https://hub.docker.com/>

Go to the repositories section.

You should see:

reactapp

The Docker image is now available on Docker Hub.

Part 10 – Share the Docker Hub Repository

Student A can share the Docker Hub repository name with Student B.

Example:

studentA/reactapp:latest

Student B does not need the original React project.

Student B only needs:

- Docker Desktop
 - Internet connection
 - The Docker Hub image name
-

Part 11 – Student B: Pull the Docker Image

Student B should install and start Docker Desktop.

Open PowerShell.

Student B can now download the image using:

```
docker pull studentA/reactapp:latest
```

Replace **studentA** with the actual Docker Hub username.

Docker will download the image from Docker Hub to Student B's computer.

Check the downloaded image:

```
docker images
```

The image should be displayed.

Part 12 – Student B: Run the Application

Student B can now run the application without having the original React source code.

Run:

```
docker run -d -p 8081:80 studentA/reactapp:latest
```

Check the container:

```
docker ps
```

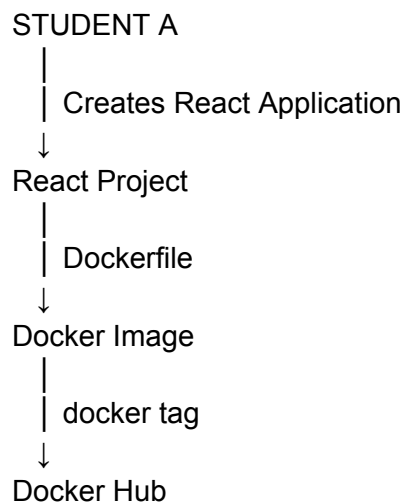
Open the browser:

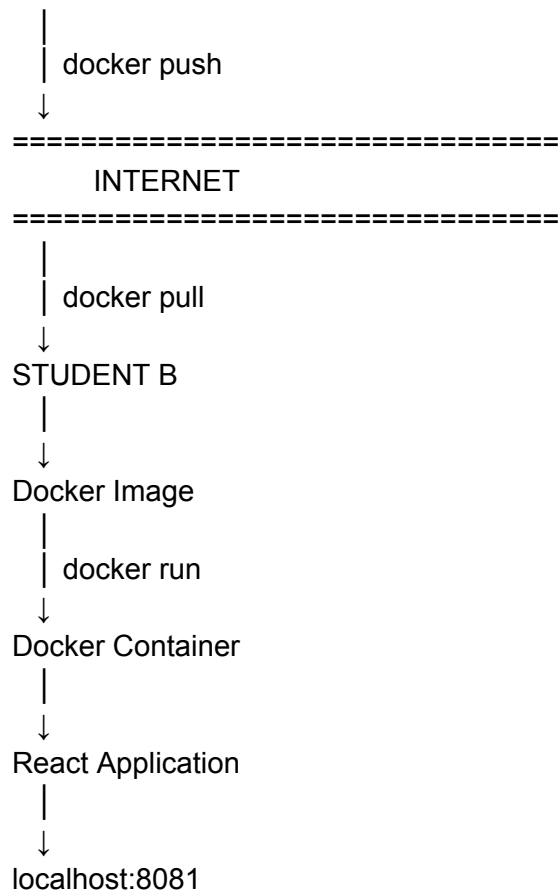
```
http://localhost:8081
```

The React application created by Student A should now be running on Student B's computer.

Part 13 – Understanding What Happened

The complete workflow is:





Part 14 – What Does Student B Need?

Student B does NOT need:

- ✗ Node.js
- ✗ npm
- ✗ Vite
- ✗ React installation
- ✗ node_modules
- ✗ Original source code

Student B only needs:

- ✓ Docker Desktop
- ✓ Internet connection
- ✓ Access to the Docker Hub image

The required application environment is already packaged inside the Docker image.

Part 15 – Public vs Private Docker Hub Repository

Public Repository

If the Docker Hub repository is public:

```
docker pull studentA/reactapp:latest
```

can be performed by other users without being granted private repository access.

Private Repository

If the repository is private, Student B must have appropriate access to the repository and authenticate with Docker Hub.

Student B can login using:

```
docker login
```

Then pull the image:

```
docker pull studentA/reactapp:latest
```

Part 16 – Useful Docker Commands

Check Docker version

```
docker --version
```

Login to Docker Hub

```
docker login
```

List images

```
docker images
```

Pull an image

```
docker pull studentA/reactapp:latest
```

Run a container

```
docker run -d -p 8081:80 studentA/reactapp:latest
```

View running containers

```
docker ps
```

View all containers

```
docker ps -a
```

Stop a container

```
docker stop <container_id>
```

Remove a container

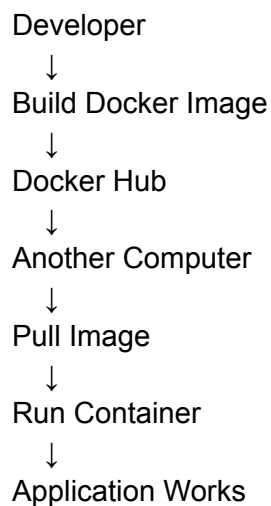
```
docker rm <container_id>
```

Final Concept

This practical demonstrates one of the major advantages of containerization:

“Build Once, Run Anywhere”

The application is packaged into a Docker image along with the required runtime environment.



The same Docker image can be distributed to multiple computers, provided that the systems support Docker and have access to the image.

Student Practical Task

Task

Student A:

1. Create a React application.
2. Create a Dockerfile.
3. Build a Docker image.
4. Run the application using Docker.
5. Create a Docker Hub repository.
6. Tag the image.
7. Push the image to Docker Hub.
8. Share the Docker Hub repository name with Student B.

Student B:

1. Install Docker Desktop.
2. Pull the Docker image from Docker Hub.
3. Run the Docker container.
4. Open the application in the browser.
5. Verify that the application works successfully.

Expected Result

Student B should be able to run Student A's React application **without having the original React source code or Node.js installed**, using only the Docker image downloaded from Docker Hub.